

Literature Review: Distributed Multi-Agent LLM Debate Systems

Team Albert Einstein | CSCI-6421 Distributed Systems | Milestone 2

1. Introduction

Large language models (LLMs) are increasingly being used as participants in structured multi-agent debate systems, where several LLM instances argue, critique, and improve their responses in multiple rounds before a final answer is generated. The motivation for this architecture is empirical: single-model outputs are well-documented to suffer from overconfidence, self-validation failures, and reasoning inconsistencies. Tillmann [Til25] has formalized the cost of naïve multi-agent deployment, documented that in fully interconnected systems. He terms phenomenon of the cost of token consumption increasing quadratically with the number of agents as context explosion, which makes the simple solution of adding more agents to the system to increase quality untenable. Meanwhile, Du et al. [Du23] demonstrate that multi-agent debate consistently outperforms single-model prompting on reasoning tasks, demonstrating that the quality gains are real and worth pursuing, provided the infrastructure cost is controlled.

The distributed systems challenge emerges at this crossroad. Current implementations of multi-agent debate, such as the four-agent LangGraph panel developed by Zarnecki [Zar25], the debate-based workflow optimizer developed by DebFlow [Su25], and the orchestrator-worker architecture developed by Magentic-One [Fou24], have all been designed as for single-machine, single-user systems. None address what happens when a worker crashes mid-debate, how debate state should be preserved across service restarts, or how to schedule hundreds of concurrent debate sessions across a cluster without starvation or resource contention. These issues define the contribution of our project: a fault-tolerant, horizontally scalable distributed system for multi-agent LLM debate.

2. Distributed Systems Challenges in Multi-Agent LLM Debate

Scalability and Context Explosion. Quadratic token growth, as described by Tillmann [Til25] is the major scalability challenge for multi-agent debate. Each agent receiving every other agent's full conversation history causes both memory and compute requirements to scale superlinearly with the number of participants. Li et al. [Li24] address this directly, demonstrating that sparse communication topologies, where each agent receives only a carefully selected subset of debate turns rather than the full transcript, achieve higher reasoning accuracy (67.5% vs. 64% on standard benchmarks) while significantly reducing token volume. This finding motivates a core constraint of any scalable design: the orchestration layer must be able to route selective message subsets, not necessarily full transcripts. Parrot [Lin24] expands upon this limitation to include the infrastructure layer, showing that LLM backend servers that treat every API call as independent and stateless leave substantial throughput on the table; on a 16-agent programming workload Parrot achieved up to 11.7× speedup over a latency-centric baseline by exploiting shared prompt prefixes and dependency-aware scheduling across requests.

Fault Tolerance and Recovery. Multi-agent debate sessions are naturally long running and stateful, with a single session potentially involving dozens or more of sequential LLM calls across several worker processes, with the output of each call informing the next. Chang and Geng [Cha25] identify this as a critical reliability gap, providing empirical results that standalone LLMs exhibit attention narrowing and self-validation failures in complex multi-step plans on the REALM benchmark. Their SagaLLM framework addresses this by treating each agent turn as a discrete SAGA transaction, with a compensating action invoked if any part of the transaction fails, returning the shared state to a consistent position. Without

equivalent durable-state infrastructure, a single worker loss in any of the single-machine debate implementations surveyed (Zarnecki [Zar25], DebFlow [Su25]) causes the entire session to be permanently lost.

State Consistency Across Distributed Workers. In a scenario with multiple agent processes sharing a common debate transcript, the problem of maintaining a consistent, linearizable view of the changing debate state is a classical distributed consistency problem. Hu et al. [Hu25] implicitly present this challenge through their adaptive stability detection mechanism, where the judge agent must read a globally consistent snapshot of all debater outputs to decide whether convergence has been reached. In a distributed setting, this read must be atomic as a partial read of the debate state mid-write would cause the stopping criterion to fire incorrectly. Magentic-One [Fou24] addresses a related consistency problem through its process ledger, a durable log maintained by the central orchestrator that allows it to reconstruct debate state after a failure and identify which sub-tasks have already been completed.

Multi-Tenancy and Scheduling. A production debate service must support many concurrent users, each triggering a separate debate session of unpredictable length and token cost. Li et al. [Li24] observe that their sparse topology results hold across different agent counts, suggesting that sessions will compete for heterogeneous resource profiles. Without a scheduling component that manages the per-session token allowance and maps sessions to free workers, the cluster will exhibit uneven load, with some workers remaining idle while others are overloaded. Parrot's semantic variables [Lin24] provide a partial solution: by annotating prompts with structured placeholders that expose the dependency and commonality structure of requests to the serving backend, the scheduler can co-locate dependent turns on the same GPU and share KV-cache prefixes across sessions running the same agent role prompts.

3. Survey of Approaches

A. Agent Role Design

The most consistent result across the literature is that differentiated agent roles outperform homogeneous roles. Three distinct design philosophies appear. Tillmann [Til25] classifies the field as prompt-engineered role separation (the same base model given different system prompts per role) versus model-family-based separation (different LLMs per role), finding that prompt engineering dominates in practice and that removing role-specific prompts causes the most substantial performance drop in ablation studies. Zarnecki [Zar25] outlines a concrete four-role implementation, with the agents designed as Researcher, Expert, Critic, Moderator. The Critic is explicitly tasked with finding flaws the Expert missed and consistently identifies compliance and cost risks that a single model with multiple prompts does not replicate. Hu et al. [Hu25] take a two-class approach that separates argument generators from a distinct LLM judge, finding that this separation produces well-defined termination behavior: the judge reads all debater outputs and decides whether stability has been reached, converging in two to seven rounds in most tested cases.

B. Communication Topologies

The way debate turns are routed between agents has also proven to be as consequential as role design. Tillmann's survey [Til25] organizes topologies along three axes: hierarchical vs. non-hierarchical, fixed vs. dynamic, homogenous vs. heterogeneous. They find that fixed topologies are better at controlling token cost than peer-to-peer alternatives. Li et al. [Li24] provide the strongest evidence so far for the use of a sparse topology, showing that agents connected in a ring outperform fully connected ones in terms of both accuracy and efficiency, with the counter-intuitive finding that restricting information flow actually improves reasoning by reducing noise propagation. Zarnecki's Moderator-controlled topology [Zar25] adds a stability

argument: the Moderator's explicit authority overturn-taking via deterministic transfer functions prevents the deadlock failure mode that flat consensus approaches can fall into. These findings converge on a shared design principle: a central authority that controls message routing is both more efficient and more stable than distributed consensus.

C. Decision-Making and Stopping Rules

Three types of stopping rules appear across the literature, each with their unique cost-quality trade-offs. The first is the simplest, which is commonly used: fixed round caps. But both Tillmann [Til25] and Hu et al. [Hu25] demonstrate they are wasteful: most debates converge well before their cap, meaning compute cycles are spent on rounds that produce no new information. Hu et al.'s adaptive stability detection attacks this problem directly, reducing average debate rounds by approximately 40% compared to fixed-cap baselines without sacrificing accuracy. DebFlow [Su25] presents a third approach: node-level error signals that identify which specific step in a workflow failed, allowing the stopping criterion to differentiate productive disagreement from mere repetition. DebFlow reaches its best-performing configuration in roughly half the optimization iterations required by single-LLM baselines, presenting that debate-driven convergence with fine-grained feedback is more computationally efficient than single-agent optimization.

D. Orchestration Systems and Infrastructure

The orchestration layer is where most existing systems tend to show their limitations. Magentic-One [Fou24] is the most architecturally well-developed system of the systems surveyed, achieving a 38% task completion rate on the difficult GAIA benchmark through a central orchestrator that maintains a process ledger, breaks down the tasks into sub-tasks handled by specialized agents, and supports replanning upon failure. Its flexible agent design where agents can be added or removed without retraining establishes the extensibility pattern our system adopts. SagaLLM [Cha25] also applies transactional discipline to orchestration, where each interaction between agents is treated as a SAGA step with a compensation plan, verified through a two-tier Global Validation Agent that checks the output of each individual agent and the messages exchanged between agents. Parrot [Lin24] addresses the infrastructure layer beneath the orchestrator, demonstrating that an LLM serving system aware of application-level request structure can schedule dependent turns consecutively and share KV-cache prefixes across sessions, achieving up to $2.45\times$ throughput improvement over naive serving baselines. Crucially, however, none of these systems tackle the full stack: Magentic-One and DebFlow [Su25] lack fault recovery; SagaLLM [Cha25] and Parrot [Lin24] lack multi-agent debate role semantics. Our project combines these layers into a single distributed architecture.

E. Taxonomy of Multi-Agent Debate Architectures

Four architectural patterns emerge from the debate literature, each reflecting a different philosophy about how agents should communicate. The oldest is flat peer-to-peer debate [Du23], where all agents are equal, fully connected, and vote on a final answer. This is prone to conformity collapse and the quadratic token growth that makes it unworkable at scale. Persona-based systems like ChatEval [Cha23] improve on this by giving agents distinct roles before the debate begins, which meaningfully increases output diversity, though the underlying flat topology remains and so do its scalability problems. Sparse graph-topology debate [Li24] takes a different angle entirely, restricting which agents can communicate with each other, this improves reasoning by reducing noise, while also cutting token cost. The most mature pattern is hierarchical debate [Zar25, Fou24], where a central orchestrator owns routing, turn-taking, and termination. This is the architecture our system builds on, not just because it performs well, but because the orchestrator becomes the natural place to plug in fault recovery, stopping rules, and multi-user scheduling.

4. Project Proposal and Connection to Prior Work

Our project builds a distributed multi-agent LLM debate service that addresses the infrastructure gaps identified in Section 2 while implementing the debate design patterns validated in Section 3. We adopt Zarnecki's four-role structure [Zar25] (Researcher, Expert, Critic, Judge) as the basis for our worker service types, with each role running as an independently scalable containerized process. We replace LangGraph's in-process function calls with durable message queue routing, preserving the hierarchical Moderator-controlled topology that Zarnecki and Tillmann [Til25] both identify as more stable than peer-to-peer alternatives, while adding the cross-process fault tolerance neither system provides.

Our stopping rule directly implements Hu et al.'s adaptive stability detection [Hu25], augmented with DebFlow's node-level error signal approach [Su25], so the orchestrator terminates a session when no substantive new content was introduced in the last round and no pending compensation actions remain. Debate state is persisted to PostgreSQL as SAGA transactions following Chang and Geng's pattern [Cha25], so that a crashed worker can be retried against a durable, consistent state snapshot. For the serving layer, we implement Parrot's semantic variable approach [Lin24] to expose turn dependency structure to the LLM backend, enabling KV-cache sharing across concurrent sessions running the same role prompts. The gap our project fills is the combination of all four concerns: role-differentiated debate, sparse hierarchical routing, adaptive stopping, and durable fault recovery in a single horizontally scalable distributed system. No existing work addresses all four simultaneously.

References

- [Cha25] Chang, E. Y., & Geng, L. (2025). SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning. *Proceedings of the VLDB Endowment*, 18(12), 4874–4886.
<https://doi.org/10.14778/3750601.3750611>
- [Du23] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). Improving Factuality and Reasoning in Language Models through Multiagent Debate. *arXiv preprint arXiv:2305.14325*.
- [Fou24] Fournay, A., Bansal, G., Mozannar, H., et al. (2024). Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. *arXiv preprint arXiv:2411.04468*. <https://doi.org/10.48550/arXiv.2411.04468>
- [Hu25] Hu, T., Tan, Z., Wang, S., Qu, H., & Chen, T. (2025). Multi-Agent Debate for LLM Judges with Adaptive Stability Detection. *arXiv preprint arXiv:2510.12697*. <https://doi.org/10.48550/arXiv.2510.12697>
- [Li24] Li, Y., Du, Y., Zhang, J., Hou, L., Grabowski, P., Li, Y., & Ie, E. (2024). Improving Multi-Agent Debate with Sparse Communication Topology. *Findings of EMNLP 2024*, 7281–7294. <https://doi.org/10.18653/v1/2024.findings-emnlp.427>
- [Lin24] Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., & Qiu, L. (2024). Parrot: Efficient serving of LLM-based applications with semantic variable. In *Proceedings of the 18th USENIX OSDI*, Article 50, 929–945.
- [Su25] Su, J., Xia, Y., Duan, Y., Du, J., Huang, J., Shi, T., & He, L. (2025). DebFlow: Automating Agent Creation via Agent Debate. *arXiv preprint arXiv:2503.23781*. Accepted at CFAgentic @ ICML 2025.
- [Til25] Tillmann, A. (2025). Literature Review of Multi-Agent Debate for Problem-Solving. *arXiv preprint arXiv:2506.00066*. University of Göttingen.
- [Zar25] Zarnecki, M. (2025). Building an AI Debate Panel: Agents that Argue and Give a Final Conclusion. *Towards AI*.
<https://pub.towardsai.net>

[Cha23] Chan, C.-M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., Fu, J., & Liu, Z. (2023). ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate. *ICLR 2024*. arXiv:2308.07201. <https://arxiv.org/abs/2308.07201>